

MOBİL ENTEGRASYON – FLUTTER VE FASTAPI

NTP II – Hafta 13



MOBİL ENTEGRASYON – FLUTTER VE FASTAPI

Bu hafta, geliştirdiğimiz backend API'yi gerçek bir mobil uygulama (Flutter) ile entegre etmeye odaklanacağız.

Modülün temel hedefi, tam yığın (full-stack) bir uygulamanın istemci (client) ve sunucu (server) bileşenlerinin nasıl çalıştığını kavramaktır.

Bu, soyut API geliştirme sürecini gerçek dünya kullanım senaryosuna taşıyarak motivasyonu artıracaktır.



ÖĞRENME HEDEFLERİ: BÜTÜNSSEL BAKIŞ

Mobil istemcinin (Flutter) bir Python backend (FastAPI) ile nasıl HTTP istekleri aracılığıyla iletişim kurduğunu görmek.
Çapraz Kaynak Paylaşımı (CORS) politikasının ne olduğunu anlamak ve API'nin mobil istekleri kabul etmesi için neden yapılandırılması gerektiğini öğrenmek.
JWT token'ının istemci tarafında nasıl güvenli bir şekilde saklandığını ve yetkilendirme başlığı (Authorization: Bearer) olarak korumalı isteklere nasıl eklendiğini kavramak.



FULL-STACK MİMARİ: TEMEL BİLEŞENLER

İstemci (Frontend): Kullanıcının etkileşimde bulunduğu arayüzdür. Bu senaryoda Flutter mobil uygulamasıdır.

Sunucu (Backend): Veri mantığını, iş kurallarını ve veri tabanı yönetimini barındırır. Bu senaryoda FastAPI API'mizdir.

İletişim: İstemci ve sunucu arasındaki tüm etkileşimler, genellikle HTTP protokolü üzerinden gerçekleştirilen istekler ve yanıtlar (Requests/Responses) şeklindedir.

NEDEN MOBİL ENTEGRASYON?

Senaryo, analistlerin 'sahada' kullanacağı bir arayüz ihtiyacına dayanmaktadır.

API'ler genellikle soyut kalır; mobil entegrasyon, geliştirilen API'nin gerçek bir cihazda veri göstererek 'hayat bulduğunu' kanıtlar.

Bu pratik adım, öğrencilerin önceki haftalarda geliştirdikleri API'nin somut bir çıktıya dönüştüğünü görmelerini sağlar, bu da en yüksek motivasyon noktasıdır.



PROJE ÇERÇEVESİ: KULLANILAN TEKNOLOJİLER

Backend: Python tabanlı, yüksek performanslı web framework'ü olan FastAPI kullanılıyor.

Frontend: Tek bir kod tabanından iOS, Android, Web ve Masaüstü uygulamaları geliştirmeye olanak tanıyan Flutter kullanılıyor.

Temel Kavram: Flutter'da yapılan `http.post` ve `http.get` çağrıları, Swagger UI veya Colab'da yapılan testlerin mantıksal olarak aynısıdır.



MOBİL İLETİŞİMİN TEMELLERİ: HTTP İSTEKLERİ

Mobil uygulamalar, veri alışverişi için temel HTTP metotlarını kullanır:
GET: Sunucudan veri almak için kullanılır (Örn: /indicators listesini doldurmak).
POST: Sunucuya yeni veri göndermek veya kimlik doğrulaması yapmak için kullanılır (Örn: /login veya /token uç noktasına giriş bilgisi göndermek).
Bu çağrıların yapısı, Hafta 5'te (OAuth2) öğrenilen API yapısıyla birebir eşleşmelidir.



FASTAPI'Yİ YEREL AĞDA YAYINLAMA GEREKLİLİĞİ

Geliştirme ortamında, FastAPI genellikle 'localhost' (127.0.0.1) üzerinde çalışır. Ancak, bir mobil cihazın (fiziksel telefon veya emulator) bu sunucuya erişebilmesi için sunucunun yerel ağdaki tüm arayüzlere yayın yapması gerekir.

Bu, uvicorn çalıştırılırken '--host 0.0.0.0' parametresi kullanılarak sağlanır.



HOST AYARI: 0.0.0.0'IN ANLAMI

0.0.0.0 adresi, sunucunun yerel makinedeki mevcut tüm IPv4 ağ arayüzlerini dinlemesi gerektiğini belirtir.

Bu sayede, aynı yerel ağdaki herhangi bir cihaz (bilgisayar, telefon, tablet), bilgisayarınızın Yerel IP adresini kullanarak API'ye erişebilir.

Komut: `uvicorn main:app --reload --host 0.0.0.0 --port 8000`.



YEREL IP ADRESİNİ BULMA

Sunucu 0.0.0.0'da dinliyor olsa bile, mobil istemcinin erişeceği adres, sunucunun çalıştığı bilgisayarın yerel IP adresi olmalıdır. Öğrenciler, işletim sistemlerinin komut satırlarını (Windows'ta ipconfig, Linux/macOS'ta ifconfig) kullanarak bu adresi bulmayı öğrenmelidir. Örnek format: `http://192.168.1.5:8000`.



FLUTTER'DA BASE URL GÜNCELLEMESİ

Flutter uygulamasının kaynak kodunda (genellikle bir 'api_service.dart' dosyası içinde), API'nin taban URL'si ('_baseUrl') tanımlanmıştır.

Bu URL, sunucu bilgisayarın yerel IP adresiyle güncellenmelidir (Örn: 'http://192.168.1.5:8000').

Not: Android Emulator kullanılıyorsa, host makineye erişim için özel olarak 'http://10.0.2.2:8000' adresi kullanılabilir.



ÇAPRAZ KAYNAK PAYLAŞIMI (CORS) NEDİR?

CORS, tarayıcıların ve mobil uygulamaların içindeki 'webview' bileşenlerinin, farklı bir 'origin'den kaynak istemesini engelleyen bir güvenlik politikasıdır. Origin (Kaynak), protokol, alan adı (domain) ve port numarasının üçlüsünden oluşur (Örn: <http://localhost:3000>).

Güvenlik Prensipleri: Varsayılan olarak, bir kaynak (örn. mobil uygulama) yalnızca kendi kaynaklarına (same-origin) güvenebilir. Başka bir kaynaktan (cross-origin) gelen isteklere izin verilmez.

CORS'UN GEREKLİLİĞİ VE İŞLEYİŞİ

Bizim senaryomuzda: Mobil uygulama (Farklı IP/Port) FastAPI API'ye (Farklı IP/Port) erişmeye çalışır. Güvenlik politikası bunu otomatik olarak engeller. Çözüm: Sunucunun (FastAPI), istemciye özel HTTP başlıkları (Access-Control-Allow-Origin gibi) göndererek 'sana güveniyorum' demesi gerekir. Bu başlıklar, API'nin bir CORSMiddleware kullanılarak yapılandırılmasıyla sağlanır.

FASTAPI'DE CORS MIDDLEWARE UYGULAMASI

FastAPI'ye CORSMiddleware eklenerek mobil uygulamanın (veya herhangi bir tarayıcının) erişimine izin verilir.

Öncelikle gerekli paketlerin kurulması gerekebilir: 'pip install fastapi-cors' (genellikle 'fastapi[all]' ile zaten gelir).

Middleware, istek sunucuya ulaşmadan önce çalıştırılan bir fonksiyondur ve güvenlik başlıklarını ayarlamak için idealdir.

CORSMIDDLEWARE YAPILANDIRMASI (KOD)

Middleware, 'FastAPI' uygulamasının tanımlanmasının hemen ardından eklenmelidir.

Bu, gerekli HTTP yanıt başlıklarının otomatik olarak eklenmesini sağlar.
Önemli kısımlar: 'allow_origins', 'allow_credentials', 'allow_methods', 'allow_headers'.

CORS PARAMETRESİ: ALLOW_ORIGINS

'allow_origins' parametresi, API'ye erişmesine izin verilen kaynakların bir listesini tutar.

'allow_origins=[*]' kullanımı, her kaynaktan gelen isteklere izin vermek anlamına gelir.

Geliştirme Ortamında (*): Test kolaylığı için yaygın olarak kullanılır.

Canlı Ortamda (Production): Yüksek derecede tehlikelidir ve asla kullanılmamalıdır.



GÜVENLİK DERSİ: WILD CARD RİSKİ

Siber güvenlik öğrencileri için kritik bir ders: 'allow_origins=[*]' ayarı, Evrensel Erişim anlamına gelir.

Bu, API'yi Siteler Arası İstek Sahteciliği (CSRF) gibi saldırılara karşı daha savunmasız hale getirebilir (yanlış yapılandırılmışsa).

Gerçekte, yalnızca uygulamanızın alan adı belirtilmelidir (Örn: '[https://benim-flutter-app-domainim.com]').

Bu kısıtlama, yalnızca güvenilir istemcilerin API'nize erişebilmesini sağlar.



DİĞER CORS PARAMETRELERİ

'allow_methods=[*]': Tüm HTTP metotlarının (GET, POST, PUT, DELETE) kullanılmasına izin verir.

'allow_headers=[*]': İstemcinin isteğinde gönderebileceği tüm başlık türlerine izin verir (Örn: Content-Type, Authorization).

'allow_credentials=True': Çerezlerin, yetkilendirme başlıklarının veya TLS istemci sertifikalarının (credentials) isteklerde kullanılmasına izin verildiğini belirtir.



YETKİLENDİRME: JWT TOKEN KULLANIMI

API'ye giriş yapıldıktan sonra, sunucu bir JSON Web Token (JWT) döndürür. Bu token, kullanıcının kimliğini ve yetkilerini kanıtlayan şifrelenmiş bir bilgidir. Token, korumalı uç noktalara (protected endpoints) yapılan her istekte sunucuya geri gönderilmelidir.



JWT DEPOLAMA: GÜVENLİK ÖNCELİĞİ

Web'de JWT genellikle 'localStorage' veya 'cookies' içinde saklanır.

Mobil uygulamalarda ise, token'ın güvenli bir şekilde saklanması için özel kütüphaneler kullanılır.

Kaynak metin, Flutter için 'flutter_secure_storage' gibi mekanizmaların kullanıldığını belirtir.

Bu yöntemler, token'ı cihazın anahtarlığında (keychain/keystore) şifreli olarak tutarak yetkisiz erişimi engeller.



KORUMALI İSTEKLER VE AUTHORIZATION BAŞLIĞI

Giriş yapıldıktan sonra, '/indicators' gibi korumalı bir uç noktaya veri çekmek için token'in gönderilmesi zorunludur.

Token, HTTP isteğinin 'Authorization' başlığına eklenir.

Format: 'Authorization: Bearer <JWT_Token_Değeri>'.

Bu, sunucunun isteği işleme almadan önce kullanıcının yetkili olduğunu doğrulamasını sağlar.



FLUTTER: LOGIN FONKSİYONUNUN İNCELENMESİ

Öğretmen, Flutter'daki 'login' fonksiyonunu öğrencilere göstermelidir. Bu fonksiyon, API'ye HTTP POST isteği gönderir. İstek gövdesi (body), Hafta 5'te FastAPI'nin beklediği 'OAuth2PasswordRequestForm' yapısına karşılık gelen 'form_data' formatında olmalıdır. Başarılı girişin ardından dönen JWT, güvenli bir şekilde depolanır.

FLUTTER: VERİ ÇEKME FONKSİYONUNUN İNCELENMESİ

Girişten sonra, '/indicators' listesini doldurmak için bir HTTP GET isteği yapılır. Bu GET isteği, depolanan JWT token'i Authorization: Bearer başlığı olarak içermelidir.

Bu adım, öğrencilerin Hafta 3'te ekledikleri IoC (Indicators of Compromise) verilerinin mobil listede görüldüğü andır.



ÖĞRENCİ GÖREVİ: ÖZET VE KONTROL LİSTESİ

1. FastAPI sunucusunu '0.0.0.0' hostu ile yerel ağda başlatın.
2. Sunucuya 'CORSMiddleware' ekleyerek mobil erişime izin verin.
3. Kendi bilgisayarınızın yerel IP'sini bulun.
4. Flutter kodundaki '_baseUrl' değişkenini bu yerel IP ile güncelleyin.
5. Flutter uygulamasını çalıştırın ve '/login' üzerinden API'ye giriş yapın.
6. '/indicators' listesinin mobil uygulamada olduğunu kontrol edin.

ADIM 1: BACKEND HAZIRLIĞI (FASTAPI)

FastAPI uygulamasını başlatan ana dosyaya ('main.py' gibi) 'CORSMiddleware' import edilir.

'from fastapi.middleware.cors import CORSMiddleware' satırı kullanılır.

Uygulama örneği ('app = FastAPI()') oluşturulduktan hemen sonra middleware ekleme işlemi yapılmalıdır.

ADIM 2: SUNUCUYU BAŞLATMA VE AĞI DİNLEME

Kullanılacak komut: 'uvicorn main:app --reload --host 0.0.0.0 --port 8000'.
'main:app': 'main.py' dosyasındaki 'app' objesini çalıştırır.
'--reload': Dosya değişikliklerinde sunucunun otomatik yeniden başlatılmasını sağlar.
'--host 0.0.0.0': Yerel ağdaki tüm arayüzleri dinlemesini sağlar, mobil erişim için kritik öneme sahiptir.

ADIM 3: İSTEMCİ KODUNUN İNCELENMESİ (FLUTTER)

Öğrencilerden sıfırdan Flutter kodu yazmaları beklenmez; önceden hazırlanmış bir proje dağıtılır.

Önemli olan, öğrencilerin Flutter'daki http çağrılarının, önceki haftalarda öğrendikleri API mantığıyla (Swagger/Colab) aynı olduğunu görmeleridir. Flutter projesi, bir 'Login' ekranı ve veri çekmeyi bekleyen boş bir 'Indicators' listesi içerir.



ADIM 4: IP ADRESİ DEĞİŞİKLİĞİ

Öğrenci, yalnızca tek bir satırı kendi ortamına göre güncellemelidir:

```
'final String _baseUrl = http://192.168.1.5:8000;'
```

Bu adresin yanlış olması veya sunucunun 0.0.0.0'da çalışmaması durumunda, mobil uygulama ağ hatası alacaktır.

ADIM 5: UYGULAMANIN ÇALIŞTIRILMASI

Öğrencilerin Flutter uygulamasını çalıştırması için Flutter kurmaları zorunlu değildir.

En kolay yöntemler:

1. Android Emulator (Host IP için 10.0.2.2 ayarına dikkat edilmeli).
2. iOS Simulator.
3. Flutter Web (Tarayıcı tabanlı çalıştırma).

KAYNAK GÜVENLİĞİ: CORS PREFLIGHT İSTEKLERİ

Bazı karmaşık veya 'güvenli' HTTP istekleri (Örn: Authorization başlığı içerenler), gerçek isteği göndermeden önce OPTIONS metoduyla bir 'Preflight' (ön kontrol) isteği gönderir.

Bu istek, sunucunun CORS politikalarını kontrol etmesini sağlar.

Eğer sunucu bu isteğe uygun başlıklarla yanıt verirse, tarayıcı/mobil istemci asıl POST veya GET isteğini gönderir.



CORS VE KİMLİK BİLGİLERİ

'allow_credentials=True' ayarı, HTTP kimlik bilgilerinin (çerezler veya Authorization başlıkları) çapraz kaynak isteklerinde gönderilmesine izin verir. JWT token'ı 'Authorization' başlığı içinde gönderildiği için bu ayar genellikle gereklidir.

Ancak bu ayar, 'allow_origins' * ile birlikte kullanıldığında bazı modern tarayıcılarda güvenlik uyarılarına yol açabilir (Güvenlik Riski).



FASTAPI'DE GÜVENLİK SINIRLANDIRMALARI

Canlı ortamda (Production), 'allow_origins' kesinlikle sınırlandırılmalıdır. CORS, sadece izin verilen istemcilerin (Güvenilir Alan Adları) API'ye erişebilmesini sağlayan bir filtre görevi görmelidir. FastAPI, bu tür güvenlik katmanlarını (Middleware) kolayca ekleyebilme esnekliği sunar.



TOKEN İŞLEME: FORM DATA VE HTTP POST

Flutter'daki login işlemi, kullanıcı adı ve parola gibi verileri HTTP POST ile gönderir.

Bu verilerin formatı, FastAPI'nin OAuth2 akışında beklediği gibi, URL kodlamalı form verisi ('form_data') olarak hazırlanmalıdır.

Bu eşleşme, Full-stack entegrasyonun başarılı olmasının anahtarıdır.

BAŞARILI ENTEGRASYONUN KANITI

Uygulama başarıyla giriş yapıp token'ı depoladıktan sonra, '/indicators' listesini çekmeli ve ekranda göstermelidir.

Bu, öğrencilerin kendi yazdıkları API'nin, gerçek zamanlı olarak mobil bir istemciye veri sağladığını gördükleri kritik başarı anıdır.

API'den gelen IoC verilerinin listede görünmesi, entegrasyonun tamamlandığını gösterir.



GELİŞTİRME MİMARİSİ: İHTİYAÇ DUYULAN BİLGİLER

Bu entegrasyon için Flutter uzmanlığı gerekli değildir.
Temel öğrenme, http çağrılarının tutarlılığı ve güvenlik mekanizmalarının (CORS, JWT) istemci-sunucu arasındaki zorunlu rolleri üzerinedir.
Entegrasyon, sadece API uç noktalarını çağırmaktan ibaret değil, aynı zamanda bu çağrıların ağ ve güvenlik kurallarına uymasını sağlamaktır.



AĞ TEMELLERİ: IP ADRESLEME VE PORTLAR

FastAPI'nin çalıştığı port (varsayılan 8000), IP adresiyle birlikte belirtilmelidir (Örn: 192.168.1.5:8000).

Mobil cihaz, bu IP adresi üzerinden yerel ağdaki sunucuya doğru porttan ulaşır. '0.0.0.0' olmadan bu iletişim gerçekleşmez, çünkü sunucu sadece kendi döngüsel arayüzünü (loopback) dinler.



JWT VE OTURUM YÖNETİMİ

JWT kullanımı, API'mizin durumsuz (stateless) olmasını sağlar. Sunucu, kullanıcı bilgilerini kendi üzerinde tutmak yerine, gelen token'ın geçerliliğini kontrol ederek yetkilendirme yapar. Bu, mobil uygulamalar gibi dağıtık istemciler için ideal bir yaklaşımdır.



MOBİL UYGULAMA GÜVENLİĞİ

JWT'nin mobil cihazlarda güvenli bir şekilde saklanması hayati önem taşır. 'flutter_secure_storage' kütüphanesi, Android'de Keystore ve iOS'ta Keychain servislerini kullanarak token'i işletim sistemi düzeyinde şifreler. Bu, diğer uygulamaların veya root erişimine sahip olmayan saldırganların token'a erişmesini zorlaştırır.



KODLAMA SÜRECİ AKIŞI: 3 SAATLİK DERSİN YAPISI

Backend Hazırlığı (60 dk): CORS middleware ekleme ve sunucuyu 0.0.0.0'da başlatma.

Frontend İncelemesi (120 dk): Flutter kodunu inceleme, IP güncellemesi, login ve veri çekme fonksiyonlarını analiz etme, uygulamayı çalıştırma ve doğrulama.

Bu yapı, pratik uygulamaya ve gözlemlemeye yeterli zaman tanır.

CORS VE TARAYICI DAVRANIŞI

Mobil uygulamalar genellikle web içeriği göstermek için 'webview' bileşenlerini kullanır.

Webview'ler, standart tarayıcılarla aynı CORS politikalarına tabidir.

Bu, mobil uygulamanın kendi kodundan yapılan doğrudan http çağrıları olmasa bile, web içeriği gösterirken CORS ayarlarının geçerli olduğu anlamına gelir.



FASTAPI VE DİĞER PYTHON FRAMEWORKLERİNDE CORS

Flask, Django gibi diğer Python framework'leri de CORS yönetimi için benzer middleware veya eklentiler kullanır.

FastAPI'nin 'CORSMiddleware' yapısı, Starlette üzerine inşa edilmiştir ve son derece hızlı ve yapılandırılabilir bir çözümdür.

Bu, modern API geliştirmede standart ve önerilen bir güvenlik uygulamasıdır.

YEREL IP ADRESİNİ BULMA YÖNTEMLERİ

Windows: Komut İstemi'nde 'ipconfig' komutu kullanılır ve genellikle 'IPv4 Adresi' başlığı altında bulunur.

Linux/macOS: Terminal'de 'ifconfig' (eski) veya 'ip addr' (yeni) komutları kullanılır.

Doğru IP'nin seçimi (genellikle 192.168.x.x veya 10.x.x.x gibi yerel ağ aralıklarında olmalıdır), mobil erişimin başarısını doğrudan etkiler.



GİRİŞ İŞLEMİNİN DETAYLARI: FLUTTER TARAFINDAKİ İŞ AKIŞI

1. Kullanıcı, kullanıcı adı ve parola ile POST isteği tetikler.
2. API bu bilgileri doğrular.
3. API, HTTP yanıtı olarak 'access_token' ve 'token_type' (Bearer) içeren bir JSON döndürür.
4. Flutter istemcisi bu token'ı alır ve güvenli depolama alanına kaydeder.
5. Token, gelecekteki tüm korumalı API çağrıları için oturum kimliği olarak kullanılır.



VERİ GÖSTERİMİ: IOS VE ANDROID FARKLILIKLARI

Gerçek cihazlar yerel IP kullanırken, Android Emulator'lar özel bir yönlendirme adresine sahiptir.

Android Emulator, '10.0.2.2' adresini kullanarak host bilgisayarın kendisine erişebilir (bu IP, emülatörün içinde tanımlıdır).

Fiziksel cihazlar ve web, yerel ağ IP'sini (örn. 192.168.1.5) kullanmaya devam eder.



CANLI ORTAM RİSK YÖNETİMİ

Eğitim amaçlı kullanılan geniş izinler (`allow_origins=[*]`), canlıya geçişte bir 'git pull' ile bile büyük bir güvenlik riski yaratır. Geliştiricinin her zaman 'varsayılan olarak reddet, açıkça izin ver' ilkesini benimsemesi gerekir.



CORS VE CACHING (ÖNBELLEKLEME)

Sunucu, CORS Preflight yanıtında 'Access-Control-Max-Age' başlığını göndererek tarayıcının/istemcinin bu CORS politikası sonucunu ne kadar süreyle önbelleğe alabileceğini belirtebilir.

Bu, tekrarlanan Preflight isteklerini azaltarak performansı artırır.



MOBİL UYGULAMA MİMARİSİNDE API SERVİSİ

Flutter uygulamasında API iletişiminden sorumlu olan 'api_service.dart' (veya benzeri) dosyası, uygulamadaki tüm HTTP isteklerini merkezileştirir. Bu katman, token ekleme, hata yakalama ve JSON serileştirme/deserileştirme gibi görevleri üstlenir. Böylece ana kullanıcı arayüzü (UI) kodunun temiz kalması sağlanır.



VERİ DOĞRULAMA VE SERİLEŞTİRME

FastAPI, gelen verileri otomatik olarak Pydantic modelleri aracılığıyla doğrular. Mobil uygulamadan gönderilen login form verisi veya IoC verisi, backend'de güvenli bir şekilde işlenir. Bu, istemci-sunucu entegrasyonunda veri bütünlüğünü sağlar.



YEREL GÜVENLİK (LOCALHOST RİSKLERİ)

Sunucuyu 0.0.0.0'da çalıştırırken, aynı yerel ağdaki (WIFI) herhangi birinin bu API'ye erişebileceği unutulmamalıdır.
Bu, genellikle bir öğrenme ortamında kabul edilebilir bir risktir, ancak büyük bir ağda dikkatli olunmalıdır.



FULL-STACK GELİŞTİRMEDE HATA AYIKLAMA

Erişim sorunları genellikle üç ana başlıkta toplanır:

1. Ağ Hatası (Sunucuya ulaşılamıyor: Yanlış IP veya 0.0.0.0 hatası).
2. CORS Hatası (Sunucu ulaşıyor ama erişimi reddediyor: Middleware eksikliği).
3. Yetkilendirme Hatası (Token eksik veya geçersiz: JWT saklama/gönderme sorunu).



BÜTÜNSEL GÖRÜŞ: PROJE MANTIĞI

Öğrenciler 5 haftadır geliştirdikleri 'soyut' API'nin, mobil cihazda somut, etkileşimli bir arayüzle buluştuğunu görürler.

Bu, teorik API bilgisini pratik mobil geliştirme ile birleştiren bir köprü görevi görür.

Entegrasyon, sadece kodlama becerisi değil, ağ ve güvenlik protokollerini anlama becerisi gerektirir.



JWT: TOKEN YAPISI VE İÇERİĞİ

JWT, üç kısımdan oluşur: Başlık (Header), Yük (Payload) ve İmza (Signature).
Başlık: Token türünü ve kullanılan imzalama algoritmasını içerir.
Yük: Kullanıcı ID'si, son kullanma tarihi (exp) gibi iddiaları (claims) içerir.
İmza: Token'ın bütünlüğünü doğrular ve sunucu tarafından gizli bir anahtarla oluşturulur. İstemci sadece token'ı iletir, içeriğini değiştirmez.



GELİŞMİŞ CORS: BİRDEN ÇOK ORIGIN YÖNETİMİ

Gerçek uygulamalarda, geliştirme, test ve canlı ortamlar için farklı kaynaklara (origins) izin vermek gerekebilir.

FastAPI'de, CORSMiddleware'ı dinamik olarak yapılandırarak, ortam değişkenlerine göre izin verilen kaynak listesini değiştirmek mümkündür.



DART VE ASENKRON İŞLEMLER

Flutter'da HTTP çağrıları, kullanıcı arayüzünü kilitlememek için daima asenkron olarak ('async'/'await') yapılır.
Bu, sunucudan yanıt beklenirken uygulamanın akıcı kalmasını sağlar.



UYGULAMA TESTİ: EDGE CASE'LER

1. Yanlış Şifre Girişi: API'den 401 Unauthorized yanıtı gelmeli.
2. Token Süresinin Dolması: Yenileme mekanizması yoksa, kullanıcı tekrar login ekranına yönlendirilmeli (Yetkilendirme başlığı gönderilse bile 401 yanıtı alınır).
3. Sunucu Kapalıyken Erişim: Ağ hatası (Connection Refused) alınmalı.

DÖNÜŞÜM: OAUTH2 VE MOBILE

FastAPI'nin OAuth2PasswordRequestForm'u varsayılan olarak 'application/x-www-form-urlencoded' formatında veri bekler.
Mobil uygulamaların HTTP kütüphaneleri bu formatı kolayca oluşturabilmelidir.
Öğrencinin bu düşük seviyeli format dönüşümünü (form_data) anlaması önemlidir.

FULL-STACK GELİŞTİRMEDE VERSİYON KONTROLÜ

Büyük projelerde, istemci ve sunucu kodlarının (API uç noktaları, veri modelleri) senkronize kalması zorunludur.

API versiyonlama (örn. '/api/v1/indicators') bu uyumluluğu yönetmeye yardımcı olur.

Bu hafta, istemcinin ve sunucunun aynı dili (protokolü) konuştuğunun görsel kanıtıdır.



CORS HATA GİDERME İPUÇLARI

Tarayıcı Konsolunu Kontrol Etme: Eğer Flutter Web kullanılıyorsa, konsol CORS hatasının hangi kaynaktan reddedildiğini gösterir.

Allow-Headers: Authorization başlığı gönderiliyorsa ve 'allow_headers' doğru ayarlanmamışsa, istek engellenir.

Preflight (OPTIONS) Yanıtını Kontrol Etme: Sunucudan gelen OPTIONS yanıt başlıklarının doğru CORS ayarlarını içerdiğinden emin olun.

BİLGİ AKIŞI: İSTEMCİDEN SUNUCUYA VE GERİ

1. Mobil cihaz (Flutter) IP:Port üzerinden sunucuya istek gönderir.
2. İstek, işletim sisteminden geçerek ağa ulaşır.
3. Sunucu (FastAPI) isteği 0.0.0.0 üzerinden alır.
4. 'CORSMiddleware' isteği işler ve geçerliyse API uç noktasına yönlendirir.
5. API (Path Operation) veriyi işler ve yanıtı geri gönderir.

DÜŞÜK SEVİYELİ AĞ AYARLARI

Android ve iOS işletim sistemleri, varsayılan olarak HTTPS (güvenli) bağlantıları tercih eder.

Geliştirme aşamasında HTTP (güvenli olmayan) kullanılıyorsa, mobil cihaz ayarlarında (Android: Network Security Config, iOS: Info.plist'te App Transport Security) özel izinler tanımlanması gerekebilir.

Bu, dersin kapsamı dışındadır ancak gerçek geliştirme için önemlidir.

TEKRAR: JWT İÇİN BEARER ŞEMASI

Bearer token şeması, OAuth 2.0'da kullanılan en yaygın kimlik doğrulama biçimidir.

'Bearer' kelimesi, token'ı taşıyan kişinin yetkili olduğunu belirtir (taşıyıcı token). Bu, sunucunun ek bir doğrulama yapmadan token'a güvenmesi gerektiği anlamına gelir (bu yüzden güvenli depolama önemlidir).



ÖĞRETMEN NOTU: ÇALIŞAN PROJE HAZIRLIĞI

Öğretmenin, ders başlamadan önce tüm backend/frontend entegrasyonunun çalıştığından emin olması gerekir. Öğrencilerin Flutter kurulumuyla zaman kaybetmemesi ve doğrudan entegrasyon başarısına odaklanması sağlanmalıdır. Bu hazırlık, dersin motivasyonel hedeflerine ulaşmasını sağlar.



CORS: YETKİLENDİRME BAŞLIĞININ ÖNEMİ

Bazı başlıklar (Content-Type, Accept) basit kabul edilirken, 'Authorization' başlığı güvenli (safe) olmayan bir başlık olarak kabul edilir. Bu nedenle, bu başlığı içeren bir çapraz kaynak isteği, otomatik olarak bir CORS Preflight isteği tetikler. FastAPI'nin 'allow_headers=[*]' ayarı, bu başlığın sorunsuz geçmesini sağlar.

GELİŞMİŞ GÜVENLİK İHTİYAÇLARI

CORS, yatay güvenlik sağlarken, dikey güvenliği sağlamak için HSTS (HTTP Strict Transport Security) gibi diğer HTTP başlıklarının kullanılması önerilir. Bu, tarayıcıları her zaman HTTPS kullanmaya zorlar ve man-in-the-middle (ortadaki adam) saldırılarını önler (Geliştirme aşamasında HTTP kullanılması bir istisnadır).



BAŞARININ ÖLÇÜTLENMESİ

Hafta sonunda öğrenciler şunları görmüş olmalıdır:

1. Kendi yerel ağlarında bir API'yi erişilebilir kılmayı.
2. Güvenlik politikası olan CORS'u aşmayı ve doğru şekilde yapılandırmayı.
3. Mobil istemcideki JWT akışının (saklama ve kullanma) arkasındaki mantığı.
4. Geliştirdikleri tüm bileşenlerin (Hafta 3-5) nihai bir ürüne dönüştüğünü.

YEREL GİBİ ÇALIŞAN UZAK SUNUCULAR

Bu hafta yerel ağda çalıştık.

Bir sonraki adım, FastAPI sunucusunu bir bulut sağlayıcısına (AWS, Azure, Heroku) dağıtmak olacaktır.

Bu durumda, mobil uygulama IP adresi yerine canlı (production) alan adını kullanır ve 'allow_origins' ayarı bu alan adına kısıtlanır.



ÖZET: HAFTA 6 TEMEL BİLGİLER

Full-Stack: İstemci (Flutter) + Sunucu (FastAPI) entegrasyonu.

0.0.0.0: Yerel ağda erişilebilirlik için zorunlu host ayarı.

CORS: Farklı kaynaklar arası iletişimi sağlayan güvenlik middleware'i.

JWT / Bearer: Kimlik doğrulaması için istemcide saklanan ve gönderilen token.



MOBİL ENTEGRASYON – FLUTTER VE FASTAPI

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

